

ECG AI Serverless

ECG arrhythmia classification system using Machine Learning and a fully serverless AWS architecture — built as a portfolio project demonstrating practical ML, frontend, and cloud infrastructure engineering.

Overview

This project detects and classifies short ECG fragments into six arrhythmia-related rhythm classes. A Random Forest model, trained on statistical, heart-rate-variability (HRV) and frequency-domain features, runs inference inside an AWS Lambda function with zero always-on infrastructure.

```
Vercel (React / Vite) → Lambda Function URL → Random Forest Model (c
```

No API Gateway, no CloudFront for the frontend, no always-on compute: the AWS backend can be created with `terraform apply` and destroyed with `terraform destroy` between demos. The UI is hosted on Vercel. See [infra/README.md](#), [DEPLOY.md](#), and [frontend/](#).

Dataset

PhysioNet ECG Fragment Database for Dangerous Arrhythmia (2022)

Classes:

- Dangerous_VFL_VF
- Special_Form_VTTdP
- Threatening_VT
- Potential_Dangerous
- Supraventricular
- Sinus_rhythm

Dataset size:

- Total ECG fragments: 1016

Class distribution:

Class	Samples
Dangerous_VFL_VF	337
Special_Form_VTTdP	72
Threatening_VT	169
Potential_Dangerous	132
Supraventricular	106
Sinus_rhythm	200

Technology Stack

Backend / ML

- Python 3.11, NumPy, SciPy (stats, FFT, R-peak detection), Scikit-Learn (RandomForest)
- AWS Lambda (ZIP deployment) with a **Lambda Function URL** (no API Gateway)
- Amazon S3 (model storage, downloaded to `/tmp` at runtime)

Frontend

- React 19 + TypeScript + Vite

- Tailwind CSS + shadcn/ui (Radix primitives)
- React Router, TanStack Query, Axios, Zod
- Framer Motion (animation), react-dropzone (file upload)
- uPlot (ECG signal — canvas-based, built for dense time series) and Recharts (probabilities, metrics, confusion matrix)

Infrastructure

- Terraform (artifacts S3, Lambda + Function URL, IAM, CloudWatch Logs)
- Frontend on Vercel (`VITE_API_URL` → Lambda Function URL)
- Model object in an external S3 bucket (not managed by Terraform)

Removed from the Lambda runtime to fit the 250 MB ZIP limit: `wfdb` (replaced by a minimal numpy format-16 reader) and `neurokit2` + `matplotlib` (HRV peaks now via `scipy`).

Feature Engineering

Statistical Features

mean, median, std, variance, min, max, range, rms, energy, peak_to_peak, waveform_length

HRV Features

mean_rr, std_rr, rmssd, min_rr, max_rr, rr_range

Signal Shape Features

skewness, kurtosis

Frequency Domain Features

dominant_frequency, spectral_energy, spectral_entropy

Total features: 22 — see backend/constants.py::FEATURE_COLUMNS (single source of truth for training and inference) and backend/features/ecg_features.py.

Model

- RandomForestClassifier (class_weight="balanced", random_state=42)
 - Accuracy: **76.96%** · Balanced Accuracy: **75.6%**
 - Metrics (accuracy, balanced accuracy, per-class precision/recall/F1, confusion matrix, class distribution) are computed by backend/models/train.py::save_metrics and persisted to data/models/model_metadata.json, which the /metrics API route serves directly. Per-class precision/recall/F1 and the confusion matrix are null until scripts/train_model.py is re-run with the raw PhysioNet dataset present (not bundled in this repo) — the /metrics route and the Model Performance page both degrade gracefully in the meantime.
-

API contract

Exposed directly on the Lambda Function URL (paths below). The Vercel frontend sets VITE_API_URL to that URL. See backend/services/lambda_handler.py.

Route	Method	Description
/health	GET	Liveness + "is the model loaded" check

/metrics	GET	Static evaluation metrics (see data/models/model_metadata.json)
/predict	POST	Runs inference on an uploaded ECG record

POST /predict request body:

```
{
  "header": "<base64-encoded .hea file>",
  "signal": "<base64-encoded .dat file>"
}
```

POST /predict response body:

```
{
  "prediction": {
    "class_id": 5,
    "class_name": "Sinus_rhythm",
    "confidence": 0.83,
    "probabilities": { "Dangerous_VFL_VF": 0.01, "...": 0
  },
  "features": { "mean_rr": 0.83, "...": 0 },
  "signal": { "sampling_rate": 250, "values": [ /* downsam
  "meta": { "inference_time_ms": 42.1, "model_version": "
}
```

Errors return {"error": "..."} with 400 for malformed/invalid input and 500 for unexpected failures (see ApiError in lambda_handler.py).

Project Structure

```
ECG_AI_Serverless/
├── backend/                                # SOURCE (development)
```

```

├── constants.py                # CLASS_NAMES, FEATURE_COLS
├── features/
│   └── ecg_features.py
├── models/
│   ├── train.py                # trains + persists data/model
│   └── predict.py
├── services/
│   ├── ecg_service.py          # WFDB reader (filesystem)
│   ├── lambda_handler.py       # HTTP router: /health, /metrics
│   ├── metrics_service.py
│   └── model_loader.py          # cross-platform /tmp caching
├── utils/
│   └── mock_ecg.py              # synthetic ECG generator
├── data/
│   ├── models/
│   │   ├── random_forest_final.joblib    # trained model
│   │   └── model_metadata.json            # served by GET
│   └── mock.hef / mock.dat                # tiny synthetic dataset
├── frontend/                          # React + Vite SPA
│   ├── public/samples/                 # synthetic per-class ECGs
│   └── src/
│       ├── app/                         # router, providers (React)
│       ├── components/                  # ui/ (shadcn primitives)
│       ├── features/                    # ecg-upload, ecg-classification
│       ├── pages/                       # Analyze, ModelPerformance
│       ├── services/api/                 # axios client + Zod schemas
│       └── types/ · utils/
├── infra/                              # Terraform: artifacts S3 bucket
├── DEPLOY.md                           # one-command AWS backend deploy
├── scripts/
│   ├── deploy.ps1 / deploy.sh          # build Lambda ZIP + terraform
│   ├── destroy.ps1 / destroy.sh
│   ├── build_dataset.py                 # re-extract features from WFDB
│   ├── train_model.py
│   ├── predict_sample.py
│   └── generate_frontend_samples.py      # synthetic per-class ECGs

```

```
|   └─ build_package.py           # builds package/ + lambda
|
|─ package/                      # GENERATED staging dir (
|─ lambda.zip                    # GENERATED deploy artifa
|─ test_lambda.py                # local end-to-end test (
|─ requirements.txt              # dev deps (pandas + boto
|─ requirements-lambda.txt       # runtime deps (numpy/sci
|─ .gitignore
```

Deployment strategy (why this fits a ZIP)

A naive vendoring of the scientific stack was **~290 MB unzipped** — over the 250 MB hard limit for `.zip` Lambda. The size came almost entirely from two optional dependencies:

- `neurokit2` (HRV R-peak detection) → pulls in **matplotlib** (~70 MB).
- `wfdb` (record reader) → pulls in matplotlib + aiohttp/requests/fsspec.

This project removes both **without changing the model's role**:

- R-peak detection uses a small **Pan-Tompkins-style detector built on `scipy.signal`** (`backend/features/ecg_features.py`). `scipy` is already a scikit-learn dependency, so this adds nothing to the package.
- ECG files are read by a **minimal numpy WFDB format-16 reader** (`backend/services/ecg_service.py`), from disk or directly from in-memory bytes (used by the HTTP handler).
- Inference feeds the model **numpy arrays**, so `pandas` is not packaged.

Artifact	Size
Unzipped package	~ 183 MB (limit 250 MB)

Zipped <code>lambda.zip</code>	~57 MB
--------------------------------	--------

Why no API Gateway

Lambda Function URLs give a direct HTTPS endpoint with zero extra cost beyond the Lambda invocation itself. API Gateway's free tier only lasts 12 months on new AWS accounts; Function URLs plus Lambda's always-free tier keep the backend near **\$0** for portfolio demos. The frontend is hosted on Vercel and calls the Function URL with CORS enabled. See [infra/README.md](#).

Local development

Backend

```
pip install -r requirements.txt
python test_lambda.py
# -> GET /health -> {...}
# -> GET /metrics -> {...}
# -> POST /predict -> {...} (synthetic ECG, no dataset re
```

Retrain (requires the raw `ECG_DB/` dataset, not included in this repo):

```
python scripts/build_dataset.py      # writes data/processed
python scripts/train_model.py        # writes the model + d
```

Regenerate the frontend's synthetic per-class samples:

```
python scripts/generate_frontend_samples.py
```


Frontend

```
cd frontend
npm install
npm run dev
```

By default `/api/*` is proxied (see `vite.config.ts`) to `http://localhost:9000`. For a deployed backend, set `VITE_API_URL` to the Lambda Function URL (see `frontend/.env.example`).

Deploying

AWS backend (recommended one-command path):

```
# Windows
.\scripts\deploy.ps1
.\scripts\destroy.ps1 -Yes
```

```
# Linux / macOS
./scripts/deploy.sh
./scripts/destroy.sh --yes
```

The deploy script builds the Lambda ZIP, ensures the model object is in S3, runs `terraform apply`, health-checks `{function_url}/health`, and prints the Function URL. Set that value as `VITE_API_URL` on Vercel. Destroy tears down Terraform-managed AWS resources (external model bucket and Vercel stay intact). Full runbook: [DEPLOY.md](#).

Infrastructure source of truth: [infra/](#) · details: [infra/README.md](#).

Free Tier notes

- `python3.11` ZIP (no container, no ECR) → no ECR storage cost.

- No API Gateway → no 12-month-only free tier cliff.
 - No CloudFront / Route 53 / ACM for the app → frontend on Vercel hobby tier.
 - 1024 MB memory + warm-start model caching in `/tmp` keeps each invocation well within the Lambda always-free tier for personal/demo use.
-

Current Status

Completed:

- Dataset generation pipeline, feature extraction (statistical, HRV, spectral)
- Random Forest training pipeline + evaluation metrics persistence
- Dependency-light Lambda ZIP (numpy WFDB reader, scipy-only HRV, no pandas at inference)
- HTTP router (`/health`, `/metrics`, `/predict`) ready for a Lambda Function URL
- React + Vite frontend: Analyze, Model Performance, How It Works, About
- Terraform infrastructure (artifacts S3 + Lambda Function URL + IAM), `terraform validate-clean`
- Frontend prepared for Vercel (`VITE_API_URL`)

Next:

- CI for automated package builds + Terraform plan checks
- Re-run training against the full PhysioNet dataset to populate the confusion matrix / per-class metrics

Deployment layer (scripts + Terraform operability) is documented in [DEPLOY.md](#).

Limitations

This is a **technical portfolio demonstration, not a medical device**. It makes no clinical or diagnostic claims, is trained on a small public research dataset (1,016 fragments), and the "Try a sample" ECGs bundled with the frontend are synthetic signals generated to illustrate each class visually — not real patient recordings.

License

Educational and research purposes.